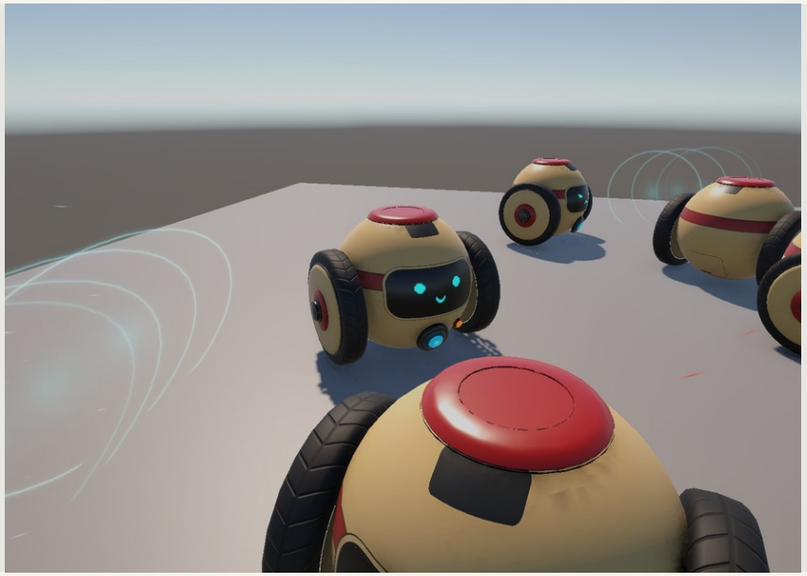
GAME CLIENT
DEVELOPMENT

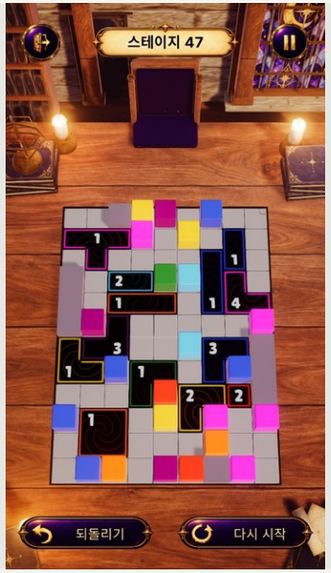
플레이를 만드는 구조,
완성도를 높이는 판단.

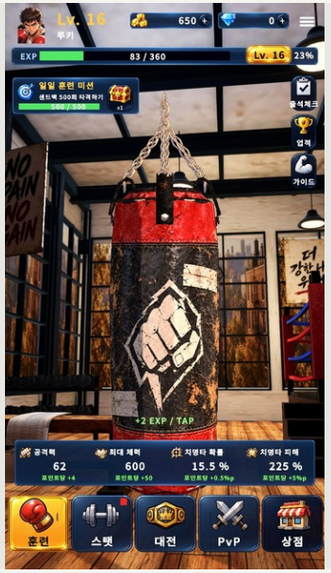
박우영 · 게임 클라이언트 개발자

wyp1226@naver.com

게임 규칙과 데이터, 제작 도구와 화면 표현을 연결합니다. AI가 제안한 결과를 프로젝트 구조와 실행 제약에 맞게 검토하고 통합합니다.



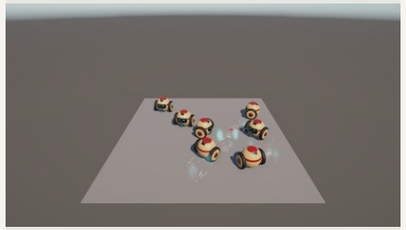




01 Lunch Royale
02 Puzzle Zip
03 Fighter Tap

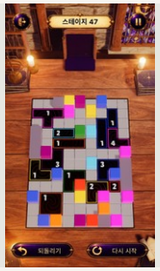
세 프로젝트로 보여주는 세 가지 개발 역량.

요청한 순서로 구성한 프로젝트와 개인 담당 범위입니다.



01 Lunch Royale

ML-Agents가 행동할 수 있는 게임 환경과 물리·판정·피드백을 구현했습니다. 2인 개발에서 클라이언트 전담. 모델 학습은 동료 AI 개발자 담당입니다.



02 Puzzle Zip (Minigame Zip)

JellyMelt와 저장·스토리·사운드·광고 시스템을 개발했습니다. 클라이언트 2명·기획자 1명의 팀에서 공용 제작 도구와 콘텐츠 확장 구조를 함께 맡았습니다.



03 Fighter Tap

게임 규칙·저장·LAN 대전부터 AI 아트의 에셋화, 3D 환경, 물리 반응, 연출까지 1인으로 구성했습니다. AI 도구는 구현·시각 제작 보조로 활용했습니다.

Lunch Royale

학습된 행동이 실제 물리와 게임 규칙 안에서 동작하도록 만든 배틀 시뮬레이션.

개발 시작 · 진행 중

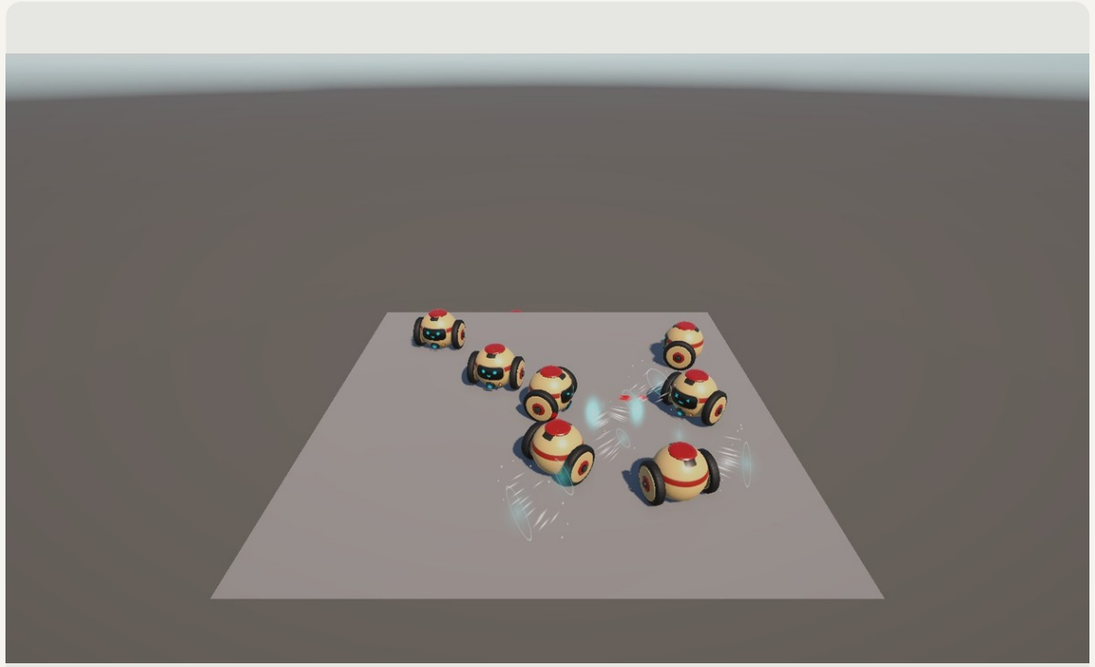
2026.06.23 시작 (최초 커밋 기준). 현재 개발 중. Unity 6000.6.6f1 / C# / ML-Agents / UniTask / URP.

팀과 담당

2명: 박우영(게임 클라이언트 전체), AI 개발자(모델 학습). 저는 관찰·행동 연결, 이동·밀치기·생사, 라운드 판정과 시각 피드백을 맡았습니다.

핵심 기여

AI 개발자와 맞닿는 입력·행동 계약을 게임 코드로 구현하고, 추론 결과가 물리·판정·표현으로 이어지도록 구성했습니다.



PLAY
실제 Play 모드 · MainScene · 2026.10.22 캡처

12
관찰값

2 + 1
연속 행동 + 이산 행동

2명
클라이언트 / AI 학습

행동 AI와 게임 시스템의 경계를 코드로 나누다.

정책의 입력과 출력, 라운드의 규칙, 화면 표현이 각자의 책임을 갖도록 구성했습니다.

ML-Agents 접점

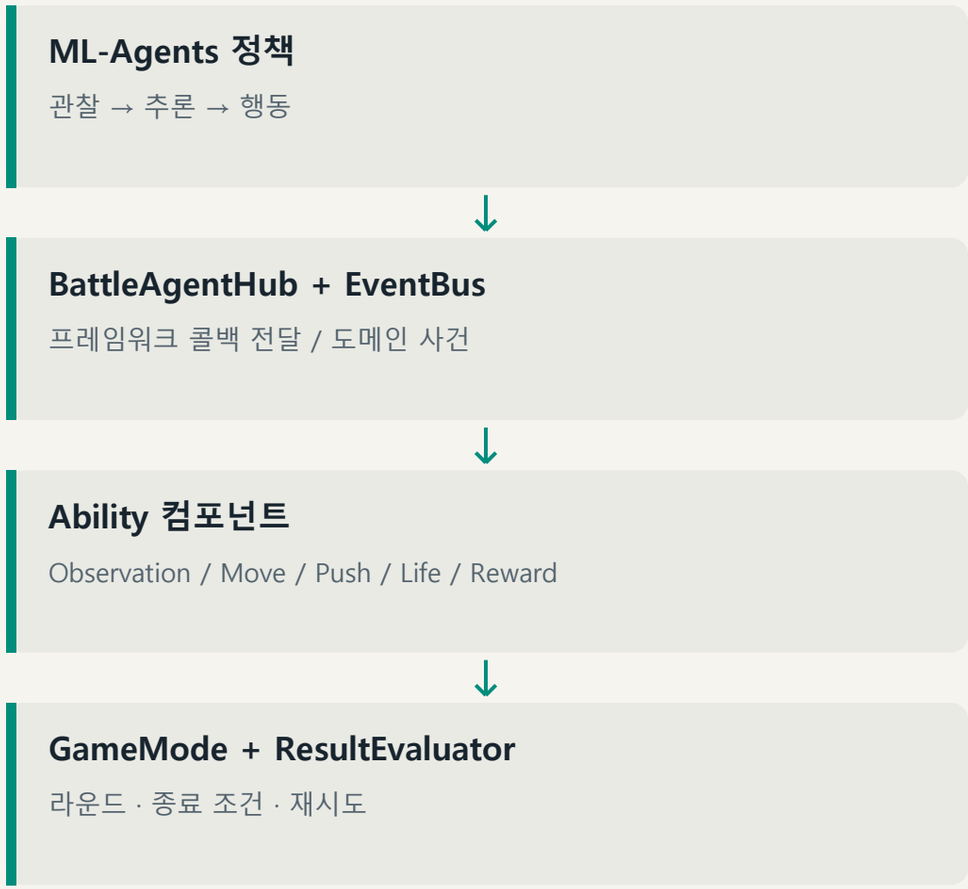
BattleAgentHub가 CollectObservations·OnActionReceived·Heuristic 콜백을 받아 Ability로 전달합니다. 학습 정책과 게임 컴포넌트 사이의 연결 지점입니다.

조합 가능한 기능

관찰·이동·밀치기·생존·보상·표정을 Ability로 나눴습니다. Hub에서 타입으로 기능을 조회하며, 피격·처치·승리 등의 사건으로 표현을 연결합니다.

별도 판정 계층

GameMode는 참가 상태와 에피소드 진행을 관리합니다. GameResultEvaluator는 ConditionGroup을 평가해 승패 또는 재시도를 알립니다.



모델을 붙이기 전에,
입력과 행동의 계약을 맞추다.

AI 학습을 직접 수행한 성과와, 학습 가능한 게임 환경을 구현한 성과를 구분합니다.

고정된 관찰 차원

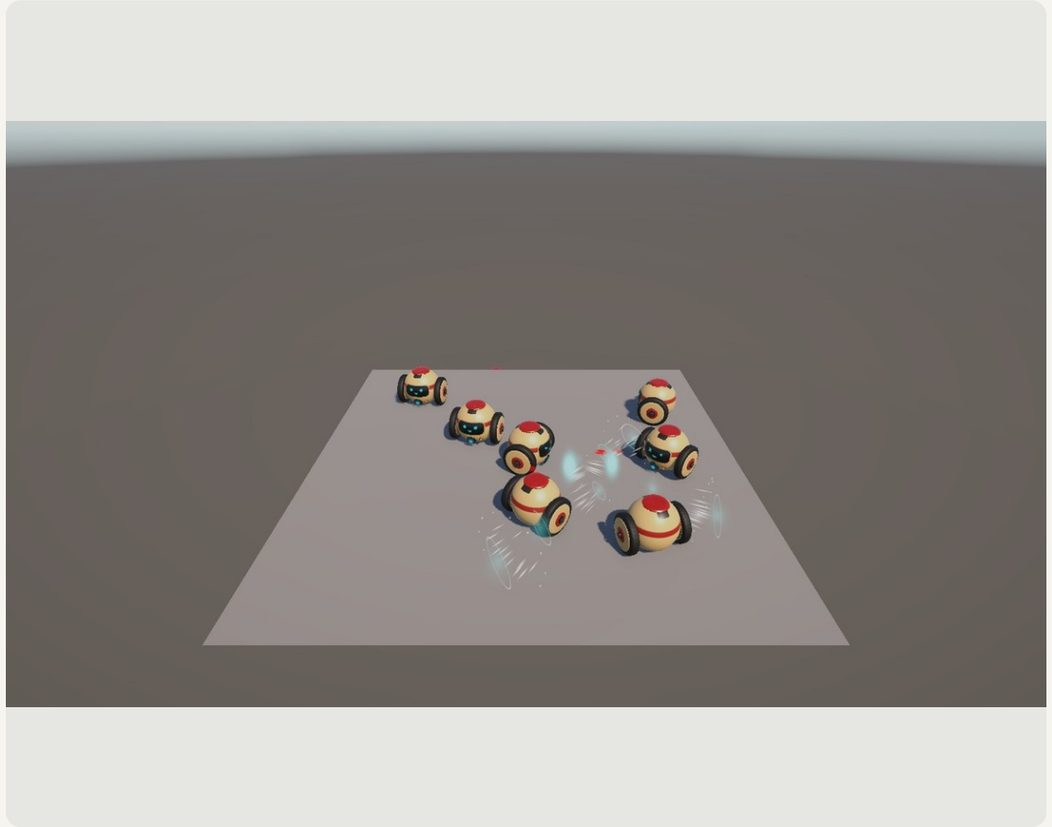
자기 위치·속도·방향 6개, 최근접 적 방향·거리·속도 5개, 밀치기 쿨다운 1개로 총 12개입니다. 적이 없거나 사망한 경우에도 0을 채워 입력 길이를 유지합니다.

물리로 연결되는 행동

연속 입력 2개는 회전·전후 이동, 이산 입력은 밀치기 실행 여부입니다. 이동은 가속·감속과 수평 최대 속도를 두고, 밀치기는 감지 대상·생존·쿨다운을 검사합니다.

협업 범위

동료 AI 개발자가 학습한 lunch_agent.onnx를 게임에서 사용합니다. 저는 관찰·행동·보상 전달 코드와 실행 환경을 담당했습니다. 학습 알고리즘·학습 성능을 제 개인 성과로 주장하지 않습니다.



PLAY
실제 Play 모드 · MainScene · 2026.10.22 캡처

6 + 5 + 1
관찰 벡터

0.5초
Push 기본 쿨다운

동시 탈락도
하나의 판정 흐름으로.

실행 순서와 생명주기를 고려한 라운드 제어 사례입니다.

같은 프레임의 사건을 수집

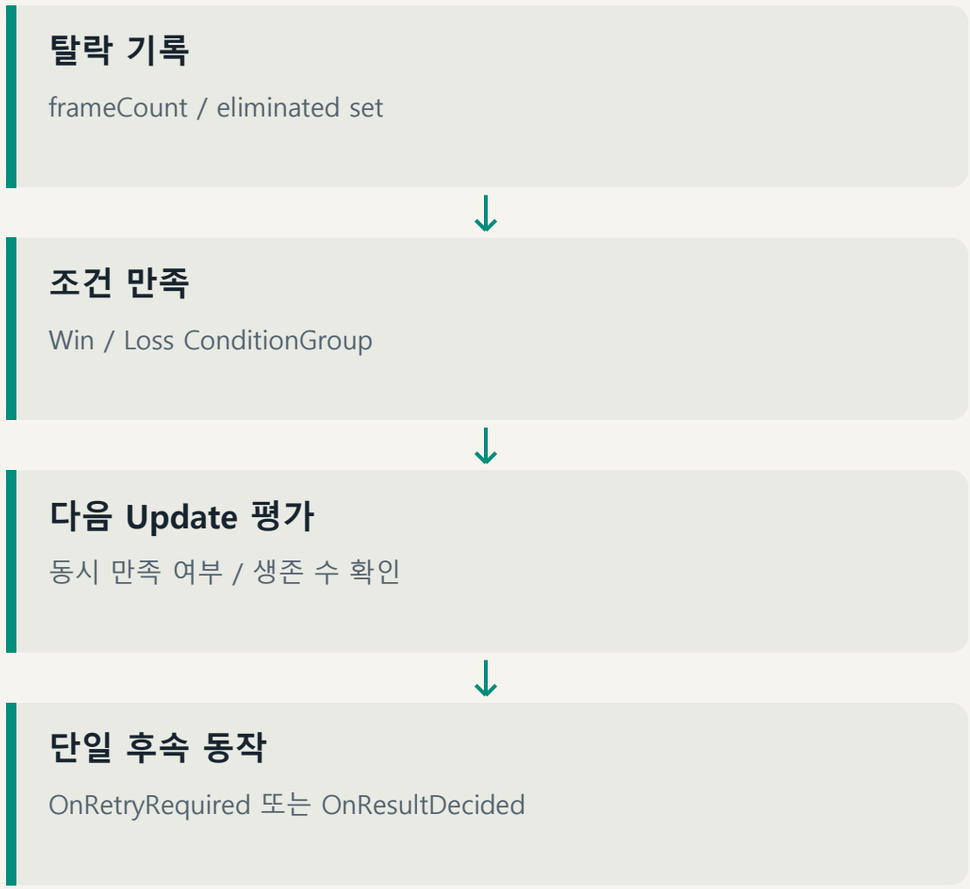
GameResultEvaluator는 탈락이 발생한 프레임과 Agent 집합을 기록합니다. 종료 조건이 만족되면 다음 Update에서 승리·패배 상태를 함께 읽어 판정합니다.

재시도와 종료를 구분

현재 구현에는 조건이 동시에 만족되고 생존자가 없는 경우, 같은 프레임에 탈락한 Agent를 다시 배치하는 경로가 있습니다. GameMode는 재시도 이벤트와 최종 결과 이벤트를 구분해 처리합니다.

비동기 수명 관리

조건 감시와 판정 대기를 연결된 CancellationTokenSource로 관리합니다. 중단·재시작 시 이전 작업을 취소해 지난 라운드의 판정이 다음 라운드에 섞이지 않도록 합니다.



AI를 사용한 방식:
제안보다 적용 기준을 먼저.

생성형 AI는 콘셉트 탐색과 구현 검토에, 학습 모델은 게임 행동에 사용했습니다.

01 게임 기능을 시각 요구로 번역

콘셉트 대화에서 둥근 본체·좌우 바퀴·도트 얼굴을 유지하고, 물리 범퍼 대신 밀치기 발사구와 동심원 충격파, 쿨다운 표시등을 요청했습니다. 기능이 보이는 캐릭터를 목표로 했습니다.

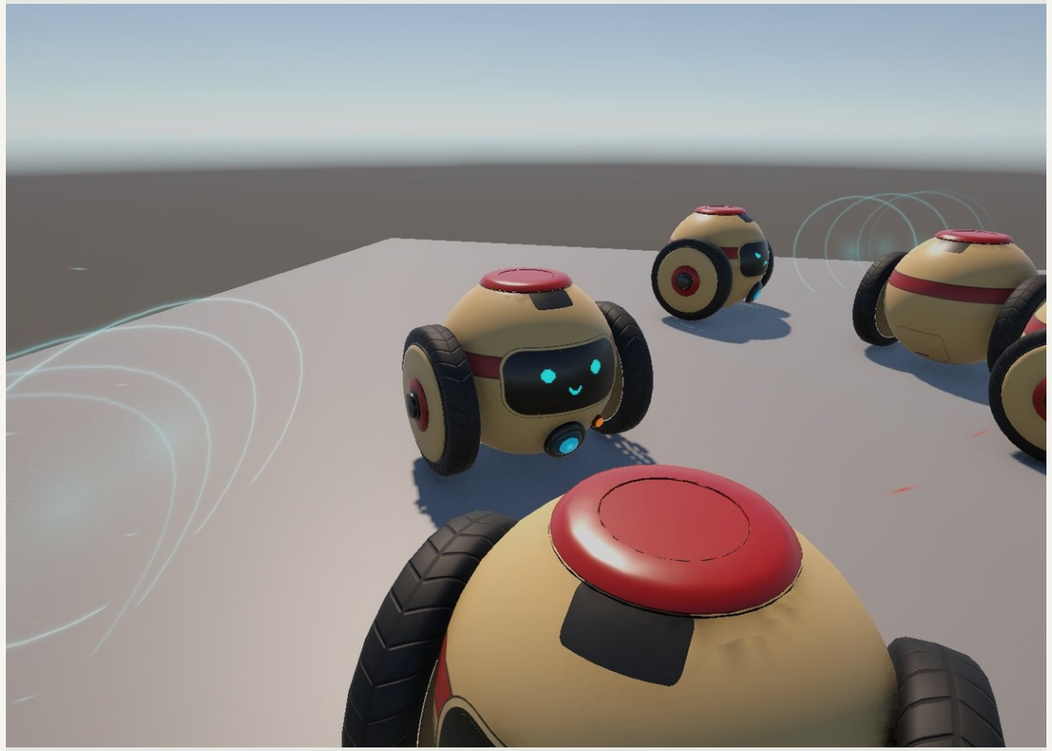
02 시안과 런타임을 분리

AI 콘셉트와 FBX 제작 탐색 기록을 참고하되, 포트폴리오에는 현재 AgentReference 모델과 실제 실행 화면을 사용했습니다. 과거 콘셉트 프리팹을 현재 적용 모델로 혼동하지 않습니다.

03 코드 제안을 현재 구조로 판단

관찰 차원 일치, Push 임시 List 할당 제거, 시간 기반 쿨다운 계산을 검토했습니다. 현재 코드는 모든 관찰 분기에서 길이를 유지하고 감지 대상을 직접 순회합니다.

도구의 제안을 게임의 입력 계약·수명·물리 규칙에 맞춰 통합합니다.



PLAY
실제 Play 모드 일시정지 · 촬영용 카메라 위치 조정 · 씬 저장 없음

행동의 결과를
표정과 충격파로 읽게 하다.

AI가 선택한 행동을 관전자가 이해할 수 있는 피드백으로 바꿨습니다.

사건 기반 표정

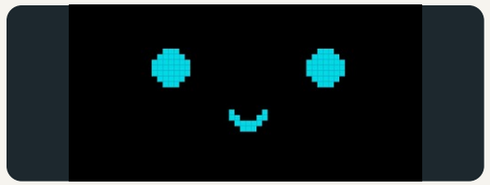
밀치기 성공·피격·처치·승리·사망을 구독해 얼굴 텍스처를 바꿉니다. MaterialPropertyBlock으로 Emission 맵을 지정하고, 짧은 반응과 라운드 종료까지 유지할 표정을 구분합니다.

반응 중첩과 초기화

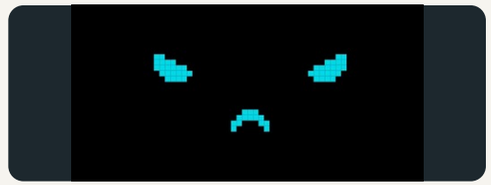
새 표정이 오면 이전 복귀 대기를 취소합니다. 사망·승리 상태는 잠그고, 에피소드 초기화와 컴포넌트 비활성화에서 원래 상태로 정리합니다.

실제 화면 확인

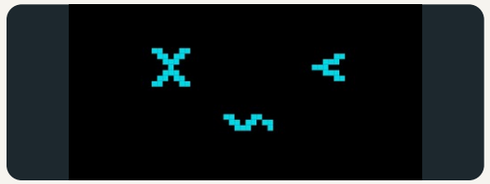
이번 포트폴리오 제작 중 MainScene에서 프레임 진행과 lunch_agent 모델 연결을 확인했습니다. 이 캡처 확인은 정책의 승률·수렴·성능 평가를 뜻하지 않습니다.



ASSET
기본



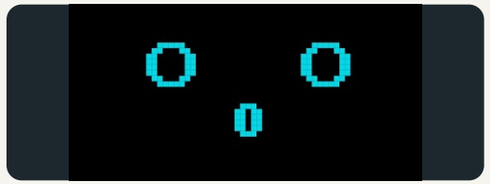
ASSET
밀치기 성공



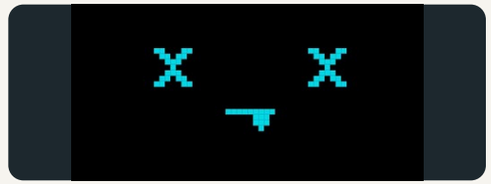
ASSET
피격



ASSET
처치 / 승리



ASSET
놀람



ASSET
사망

외부 데이터도
게임에 맞는 흐름으로.

참가자 명단을 다루는 Google Sheets 준비 계층을 개발했습니다.

다운로드와 파싱의 경계

GoogleSheetService는 URL에서 문서 ID를 추출해 XLSX를 다운로드합니다. 잘못된 URL·요청 실패·빈 응답을 검사하고 GoogleWorkbook으로 데이터를 넘깁니다.

선택과 UI 책임 분리

현재 소스에는 시트·항목 선택과 URL 입력·오류 표시 컴포넌트가 있습니다. 작업 대화에서도 패널·로딩·입력·오류·시작 버튼 책임을 나누는 방향으로 구조를 조정했습니다.

연결 범위를 명확히

현재 저장소에는 GoogleScene이 남아 있지 않고, 시작 버튼 이후 경기 씬 전환 완료 근거도 없습니다. 따라서 데이터 준비 계층 경험으로 제시하며 명단→경기 자동 연결 완료로 확대하지 않습니다.



Puzzle Zip

서로 다른 퍼즐을 공용 시스템과 제작 도구로 묶은 미니게임 프로젝트.

개발 시작 · 진행 중

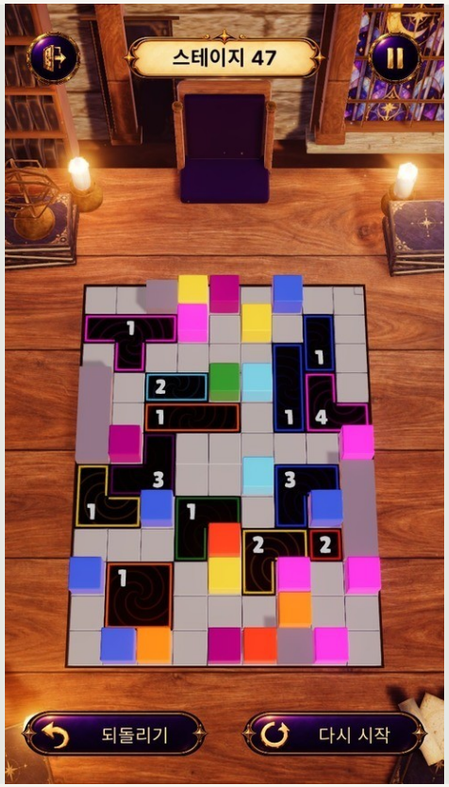
2026.04.30 시작 (최초 커밋 기준). 현재 개발 중이며 최근 AI 환경·UI 작업을 포함합니다. Unity 6000.4.0f1 / C# / UniTask / uGUI / URP.

팀과 담당

3명: 클라이언트 2명·기획자 1명. 저는 JellyMelt, 공용 저장 구조, 스토리 그래프 편집기, 사운드·광고 시스템과 최근 JellyMelt 시각 개선을 맡았습니다.

협업 기여 구분

PotionSort의 붓기·레이어 규칙 주 구현은 동료 개발자 영역입니다. 저는 공용 GameMode와 진행·UI 등 연결부를 함께 정리했습니다.



EDITOR
Stage 47 · 메시와 Canvas를 합성한 Editor 검증 렌더



EDITOR
클리어 패널 · Editor 검증 렌더

3명
클라이언트 2 / 기획 1

10개
개별 생성 UI 스프라이트

24조합
최종 UI 프레이밍 검사

퍼즐을 늘릴수록
공용 기반의 가치가 커지도록.

개별 퍼즐 로직과 저장·스토리·오디오 등 공용 시스템을 함께 개발했습니다.

JellyMelt: 데이터에서 맵으로

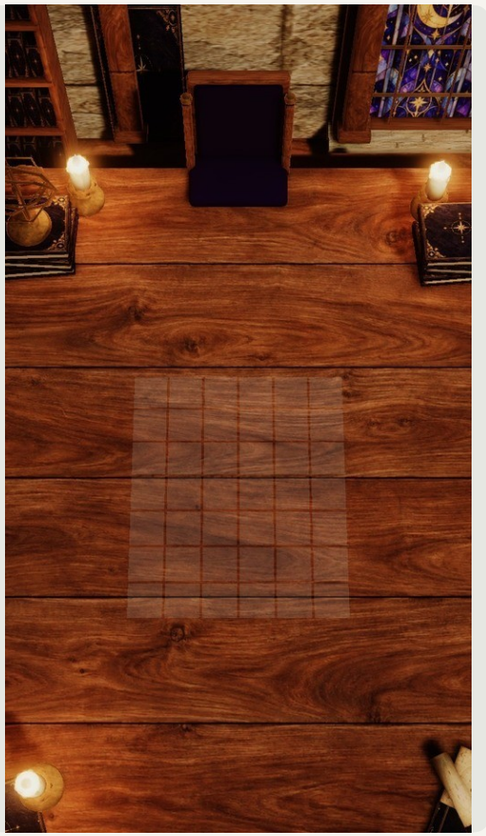
MapData의 가로·세로·타일 크기로 맵 중심을 계산하고, 바닥·고정 셀을 생성합니다. 프리팹별 풀과 활성 인스턴스를 관리해 스테이지를 바꿀 때 오브젝트를 재사용합니다.

반복 작업에서 발생한 문제 해결

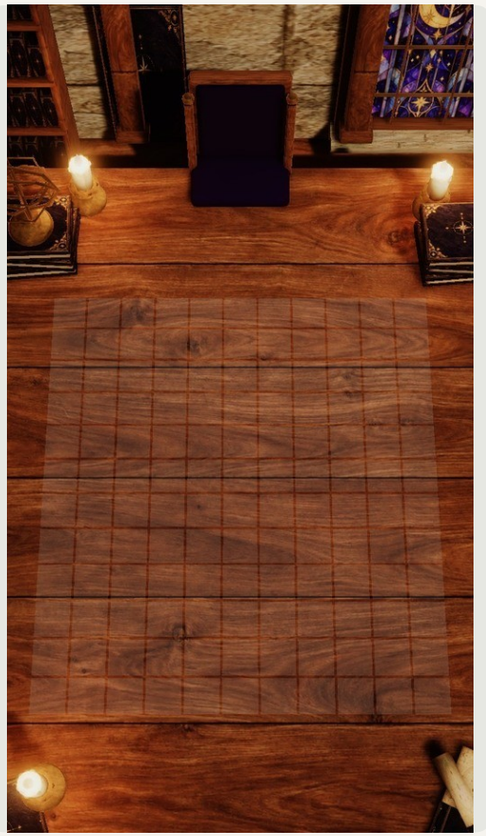
맵 초기화 시 풀링된 오브젝트 중복 반환을 수정하고, 타일 드래그와 모서리 충돌을 개선했습니다. 기능 추가와 함께 스테이지 전환의 상태 정리를 다뤘습니다.

공용 기능과 콘텐츠의 접점

게임별 성공 조건은 공용 GameMode와 연결하고 진행도·클리어 UI를 공유합니다. 사운드 ID 자동 생성·검증 도구와 광고 제어·UI를 구현해 콘텐츠 제작 흐름을 지원했습니다.



EDITOR
6×7 맵 · 환경 V2 검증



EDITOR
11×12 맵 · 환경 V2 검증

저장 형식과 게임 규칙을 서로에게 묶지 않다.

Domain / DTO / Repository 경계를 두고 설정과 게임 진행도를 관리했습니다.

게임이 다루는 것은 Domain

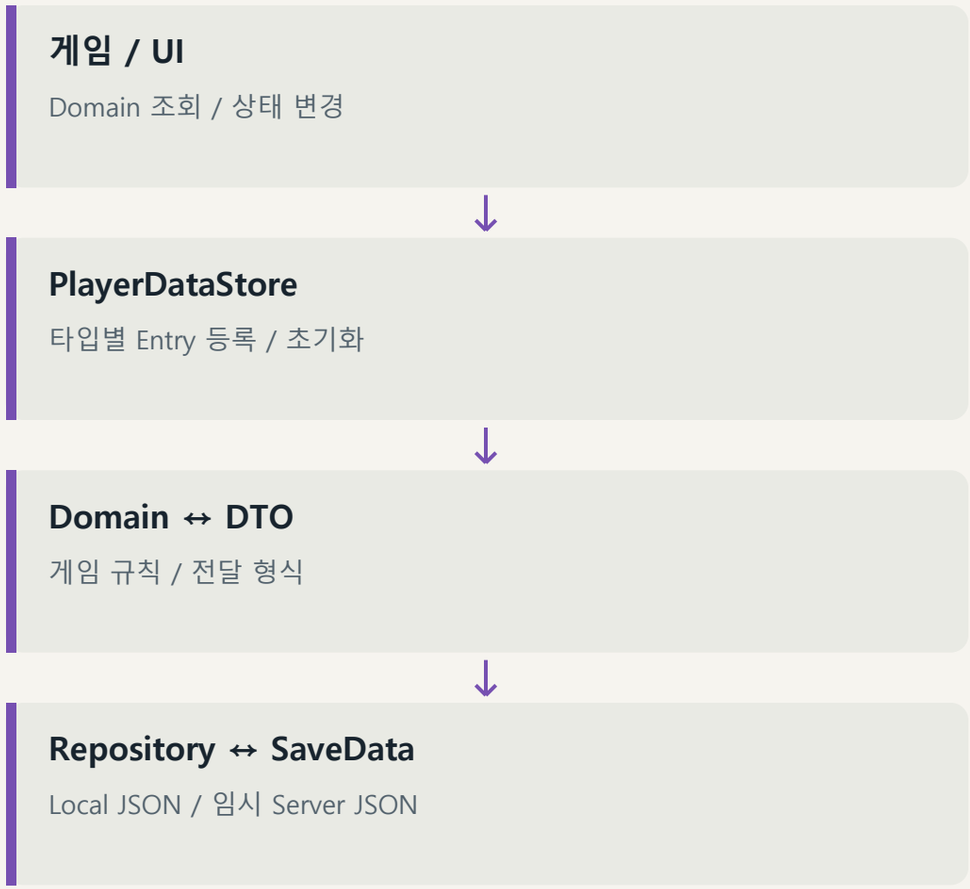
PlayerDataStore가 도메인 타입으로 Entry를 등록합니다. 게임 코드는 TryGetDomain으로 도메인을 받고 Save 또는 SaveAll로 저장을 요청합니다.

변환과 입출력은 Repository

DTO·SaveData를 구분하고 Repository가 파일 저장과 변환을 맡습니다. 설정과 게임 진행도는 같은 등록·로드·저장 흐름을 사용합니다.

확장점과 현재 구현을 구분

현재 소스의 ServerRepository도 Server 폴더에 JSON을 쓰는 임시 구현입니다. 인터페이스 분리 경험을 보여주되, 클라우드 저장이나 서버 동기화를 완료한 것으로 표현하지 않습니다.



기획 데이터를
만드는 사람의 작업까지.

스토리 그래프와 에셋 관리 도구를 구현해 콘텐츠 편집 경로를 만들었습니다.

그래프와 채팅 미리보기

StoryGraphWindow·GraphView와 노드·엣지, StoryChatView를 구성했습니다. 챕터·에피소드·메시지 단위의 데이터를 시각적으로 편집하는 도구입니다.

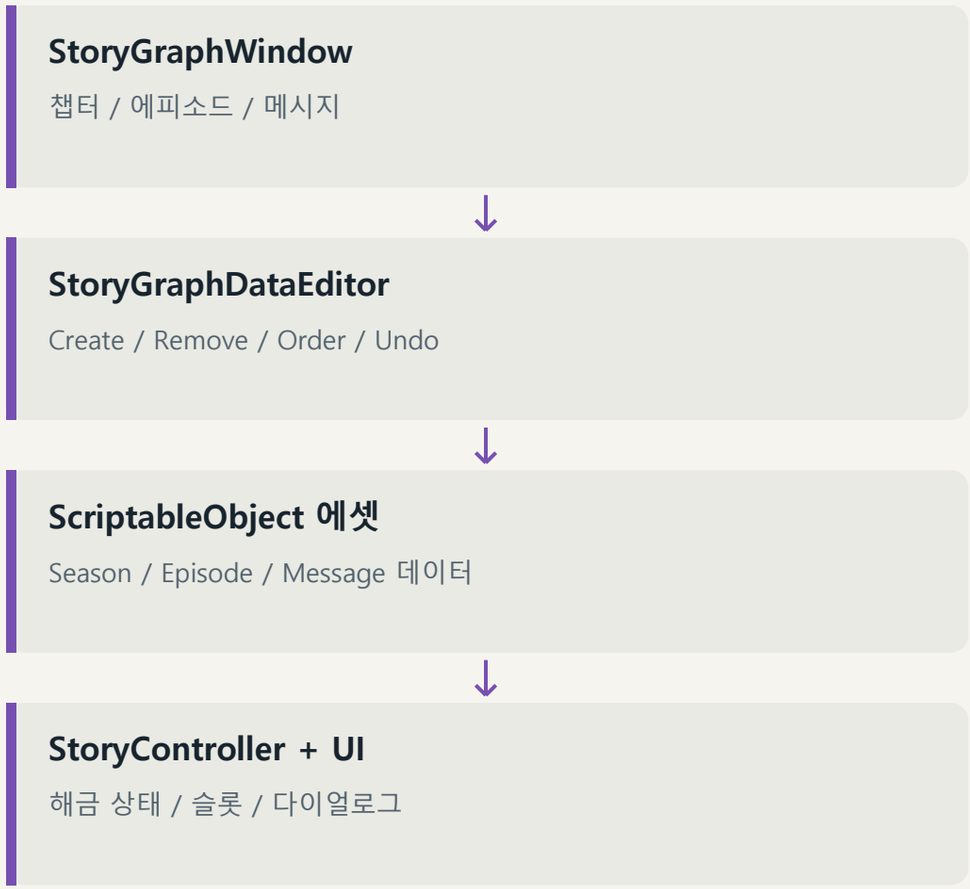
실제 에셋 구조와 연결

StoryGraphDataEditor는 선택한 저장 루트 아래 Season/Episode 에셋을 생성합니다. Undo 등록, 정렬 순서, 노드 위치 저장을 지원해 데이터와 편집 화면의 상태를 관리합니다.

플레이 UI까지 갱신

스토리 슬롯 갱신과 다이얼로그 레이아웃을 개선했습니다. 편집기 기능을 추가하는 데서 끝내지 않고, 변경된 콘텐츠가 플레이 UI에 반영되는 연결을 다뤘습니다.

플레이 기능과 제작 도구를 함께 다루는 클라이언트 개발자입니다.



AI 레퍼런스에서
가변 맵을 담는 공간으로.

시안의 분위기와 실제 퍼즐의 배치 제약을 함께 고려했습니다.

출발점: 사용자 제공 이미지

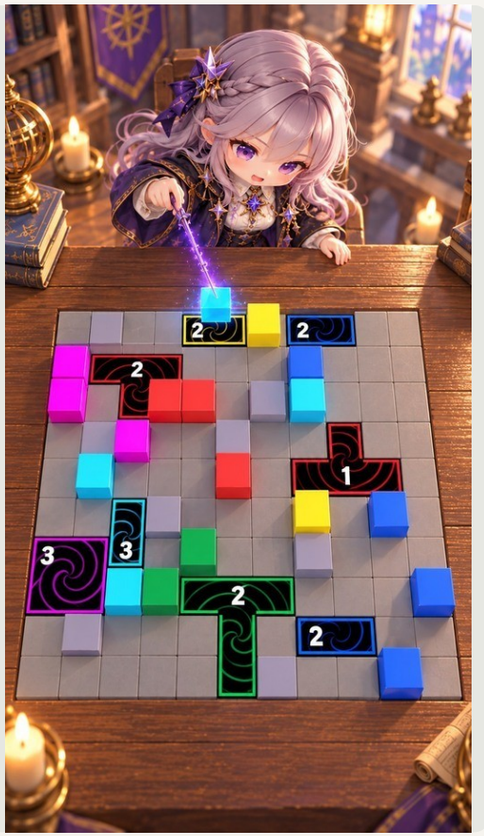
마법학교의 책상·서가·촛대·천구의를 방향으로 삼았습니다. 레퍼런스의 캐릭터와 퍼즐 이미지를 완성 화면처럼 복제하는 대신, 기존 퍼즐이 놓일 환경을 제작했습니다.

3D를 선택한 이유

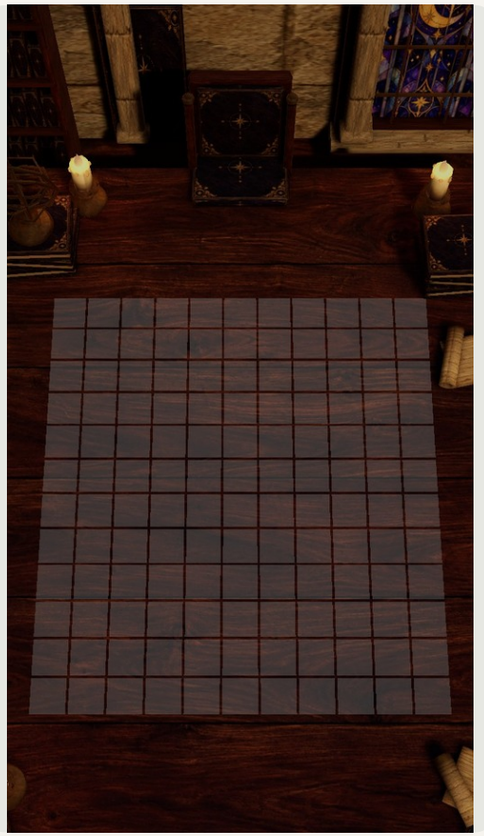
맵 크기가 계속 바뀌므로 퍼즐과 소품의 공간 경계를 조절해야 했습니다. UModeler X 원본을 만들고 Blender에서 메시·노멀을 정리한 뒤, Unity에서 소품 계층과 재질·카메라를 구성했습니다.

편집 가능한 구조를 유지

환경 V2는 24개 활성 소품 그룹과 71개 부품 메시로 구성됐습니다. 책 Cover/Pages, 촛대 Holder/Wax/Wick/Flame처럼 편집 단위별로 나눠 후속 배치 수정이 가능하도록 했습니다.



REF
사용자 제공 방향 레퍼런스 · 실제 게임 화면 아님



EDITOR
환경 V2 · 조명 베이크 단계

AI 이미지를
게임용 자산으로 바꾸는 과정.

형식과 사용 조건을 먼저 정한 뒤, 결과를 실제 에셋에 연결했습니다.

환경 텍스처: 표면 역할 지정

목재·소품 아틀라스·스테인드글라스·양피지 이미지를 개별 생성했습니다. 목재는 대비를 낮추는 추가 편집을 거쳤고, 재질별 표면 차이가 읽히도록 배치했습니다.

UI: 한 화면에서 개별 부품으로

AI UI 목업을 먼저 검토한 뒤 버튼·패널·아이콘 10개를 투명 PNG로 따로 생성했습니다. 정면 구도, 빈 중앙 영역, 작은 화면 가독성, 무문자 조건을 프롬프트에 명시했습니다.

Unity에서 완성할 부분을 남김

한글·숫자는 TMP로 렌더하고, UI 부품은 씬 인스턴스 오버라이드로 연결했습니다. 실제 데이터와 조건부 스토리 안내를 기존 컴포넌트가 표시하게 했습니다.



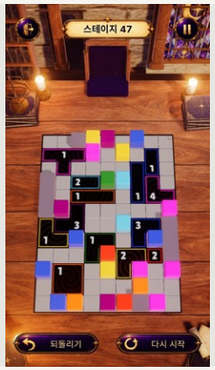
AI
생성된 소품 표면 아틀라스 · 실제 모델 적용 자산



AI
생성·수정된 목재 텍스처 · 최종 V2 자산



AI
AI UI 시안 · 실제 실행 화면 아님



EDITOR
Stage 47 · 메시와 Canvas를 합성한 Editor 검증 렌더

프롬프트도
구현 명세처럼 작성하다.

실제 저장된 생성 프롬프트의 조건을 요약한 제작 사례입니다.

입력 조건

"마법학교 모바일 UI / 정면 직교 / 보라 에나멜과 금색 테두리 / 실제 투명 알파 / 전체 실루엣 / 중앙 비움 / 글자·숫자 없음"을 지정했습니다. 아이콘은 40px에서도 구분되는 단순한 실루엣을 요구했습니다.

생성 이후의 개발 작업

스테이지 명판·원형 버튼·액션 버튼·클리어 패널에 각각 용도를 부여했습니다. 아이콘과 배경을 분리하고 텍스트는 런타임 값으로 남겨 재사용 범위를 넓혔습니다.

가져오기와 예외 처리

헤더·패널은 최대 1024, 아이콘은 256, ASTC 4×4로 설정했습니다. 아틀라스 강제 패킹 중 크래시가 발생해 미사용 아틀라스를 제외하고 개별 스프라이트 상태로 복구·검증했습니다.



같은 스테이지,
같은 카메라에서 비교하다.

Stage 47 · PC 품질 · 동일 구도의 작업 전후입니다.

변경한 표현

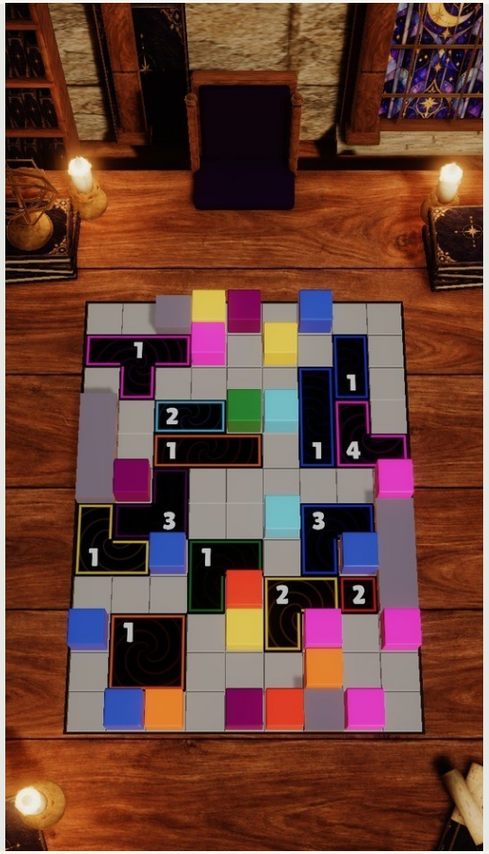
어두운 영역을 밝히고 보랏빛 그림자와 따뜻한 촛불을 더했습니다. URP PBR 기반 AcademySoftLit과 Bloom·ACES를 사용하고, 퍼즐 선명도를 위해 DOF는 껐습니다.

게임 기능과 표현의 연결

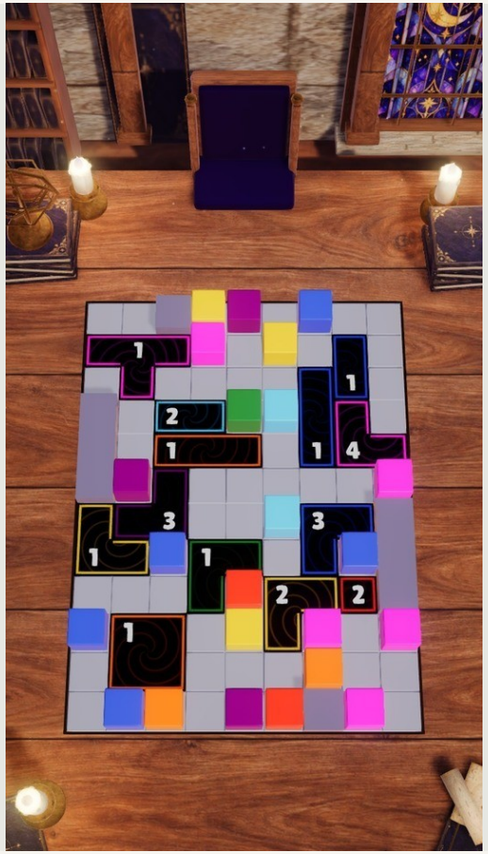
블록의 기존 색상 변경과 스킨 메시 구조를 유지했습니다. 가변 맵 생성과 불투명 바닥·외곽 프레임을 보존하고, 카메라에 상하 UI 여백을 각각 12% 확보했습니다.

비교의 기준

아래는 생성 시안과 게임의 비교가 아니라, 실제 저장된 동일 스테이지의 Editor 렌더 비교입니다. 프레임 속도 향상을 보여주는 자료는 아닙니다.



EDITOR
BEFORE · Stage 47 · 마법학교 스타일 적용 전



EDITOR
AFTER · Stage 47 · 동일 PC 품질·카메라의 스타일 적용 후

한 장의 시안보다
여러 화면 조건을 통과시키다.

작은 맵·큰 맵, 긴 휴대폰·태블릿에서 화면 경계를 검사했습니다.

프레이밍 검사

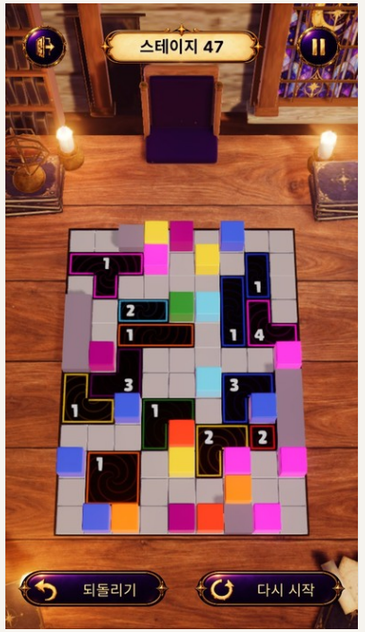
최종 UI 단계에서는 원근/직교 × 화면 비율 3종 × 맵 4종, 총 24개 조합을 확인했습니다. 환경 V2 단계의 32개 카메라 검사와는 별도 검증입니다.

비용을 고려한 환경

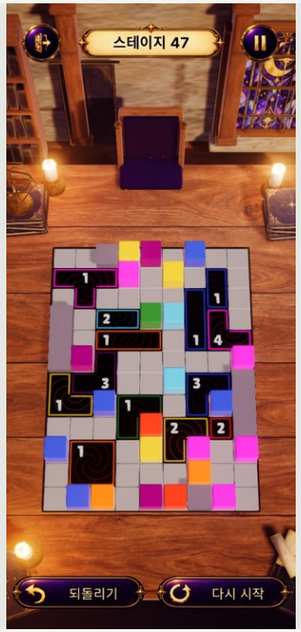
최종 환경은 72 MeshRenderer / 29,002삼각형 / 공유 재질 13개입니다. 1024 라이트맵 1장과 128 반사 프로브, 베이크 촛불·창문 조명을 사용합니다. 이 수치는 퍼즐과 UI를 제외한 환경만의 값입니다.

검증 범위

PC/Mobile 품질·UI 레이캐스트·프레이밍을 Editor에서 확인했습니다. 되돌리기·일시정지는 디자인만 적용된 상태이며, 전체 입력 흐름·모바일 실기 FPS·메모리는 추가 검증 대상입니다.



EDITOR
Mobile 품질 · Editor 렌더 · 실기기 캡처 아님



EDITOR
720×1560 비율 · Editor UI 배치 검증



EDITOR
태블릿 비율 · Editor UI 배치 검증

24
UI 프레이밍 조합

29,002
최종 환경 삼각형

Fighter Tap

탭 입력의 손맛을 성장·전투·저장·네트워크와 연결한 1인 개발 프로젝트.

개발 시작 · 진행 중

2026.10.15 시작 (최초 커밋 기준). 현재 개발 중이며 조명·물리·표현 개선을 이어가고 있습니다. Unity 6000.6.6f1 / C# / FishNet / UniTask / DOTween / Feel / URP.

담당 범위

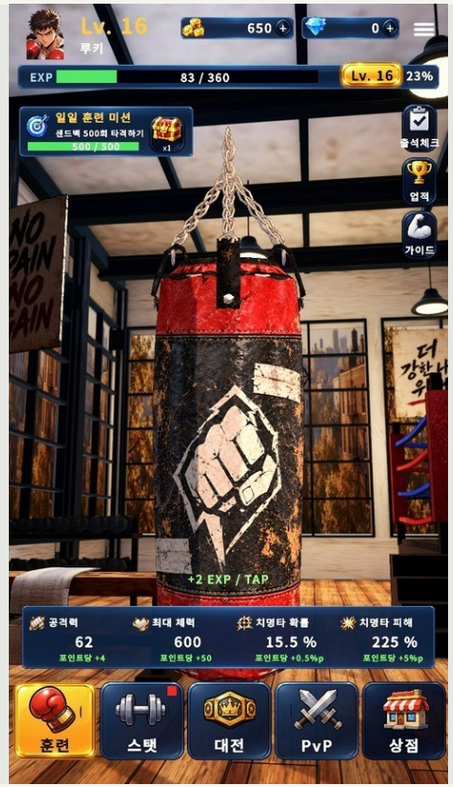
기획과 게임 클라이언트 전체, 성장·저장·NPC·LAN PvP, UI·아트 통합과 3D·연출을 담당했습니다. 생성형 AI와 코딩 에이전트로 제작을 보조하고 결과를 검토·통합했습니다.

플레이 구조

타이틀 → 게스트 시작 → 훈련 → 스탯 투자 → NPC 도전 / LAN PvP. 성장 규칙을 ScriptableObject 밸런스로 조절하고, 표현은 게임 이벤트를 구독해 실행합니다.



EDITOR
Unity 타이틀 씬 미리보기



PLAY
로비 · PC 렌더러 · HUD 포함 Play 모드 캡처

1인
게임 전체 개발

FishNet
서버 권한 LAN PvP

94.9%
샌드백 삼각형 감소

입력은 요청하고,
판정은 서버가 책임진다.

로컬 성장 게임과 LAN 대전의 규칙을 같은 전투 흐름 위에 구성했습니다.

서버가 검증하는 탭

클라이언트는 타격을 요청하고 서버가 탭 간격·성장 수치·피해·라운드를 처리합니다. 상대가 보낸 피해량·승패를 그대로 채택하지 않고 SyncVar와 ObserversRpc로 상태·표현을 전달합니다.

성장과 보상 규칙

경험치·레벨·포인트·첫 승리 보상·다음 상대 해금을 분리했습니다. 재대전에서 첫 승리 보상이 중복 지급되지 않도록 하고, 패배해도 기존 성장은 보존합니다.

저장의 실패 경로

임시 파일 작성 후 교체하고 .bak을 남깁니다. 손상 원본을 보관하고 유효한 백업을 복구하며, 미래 버전 데이터는 덮어쓰지 않습니다. LAN은 호스트를 신뢰하는 구조이며 2기기 종합 검증은 남아 있습니다.

클라이언트
Tap 요청

서버
쿨다운 / 프로필 / 피해 / 승패

상태 동기화
SyncVar / ObserversRpc



EDITOR
Unity Game View · PvP UI 표시 검증

AI에도 역할과 완료 조건을 나누다.

개발자는 구현 경계와 채택 기준을 정의하고, 도구에는 구체적인 작업 단위를 전달했습니다.

요구를 구체적인 제약으로

대상 씬과 레퍼런스뿐 아니라 기존 이벤트·참조 보존, 움직이는 물체 여부, 편집 원본 보관, 모바일 표현 비용과 완료 검사를 함께 지정했습니다. 정적 체육관과 물리 샌드백에 다른 제작 기준을 적용한 이유입니다.

기능과 시각 제작의 역할 분리

프로젝트에는 Codex가 아트·연출을, Claude가 기능·검사 도구를 보조하는 협업 지침을 뒀습니다. 체육관·경기장의 보조 C# 도구는 Claude로 작성하고, 이미지 생성과 모델·재질·연출 작업을 별도로 연결했습니다.

결과를 검토할 수 있는 형태로

프롬프트, 편집 원본, 런타임 FBX·텍스처, 검증 JSON과 전후 캡처를 남겼습니다. 저는 시각 방향과 채택 범위, 기존 기능 보존과 최종 통합을 판단했습니다. 기능 검사와 실기 성능 검증은 각각의 완료 범위로 관리했습니다.

사람의 요구와 판단

씬 / 유지 조건 / 채택 기준

↓

AI 작업 단위

기능·검사 ↔ 이미지·모델·연출

↓

프로젝트에 통합

이벤트 / 프리팹 / 에셋 참조

↓

실행·산출물 검토

검증 기록 / 전후 비교 / 남은 과제

AI 한 장을
씬 전체로 쓰지 않다.

제공된 레퍼런스를 배경·캐릭터·로고·UI로 나눠 실제 씬을 구성했습니다.

01 레퍼런스의 역할을 분해

타이틀·로그인과 훈련·스탯·대전·결과 화면을 분석했습니다. 캐릭터, 배경, 패널, 버튼 장식은 이미지로 만들고, 현재 수치와 버튼 동작은 Unity UI로 구성했습니다.

02 독립된 자산으로 생성

배경은 문구 없는 공간, 캐릭터는 투명 배경의 포즈, UI는 텍스트와 분리된 장식으로 생성했습니다. 크기·배치·레이캐스트와 상태 전환은 씬과 프리팹에서 관리했습니다.

03 실제 화면에서 확인

타이틀과 로그인 미리보기, 스탯·승리·PvP 상태를 검증했습니다. 일부 화면은 표시 이벤트를 주입한 캡처이며, 소셜 로그인·상점 등은 동작 구현 없이 표시만 있는 요소입니다.



REF
제공된 타이틀 방향 레퍼런스 · 실제 실행 화면 아님



AI
분리 생성한 타이틀 배경 자산



AI
분리 생성한 타이틀 캐릭터 자산



EDITOR
Unity 타이틀 씬 미리보기

정적 공간은 단순하게,
움직이는 물체는 별도로.

같은 AI 제작 파이프라인에도 사용 목적에 따라 다른 최적화 기준을 적용했습니다.

배경 제작 파이프라인

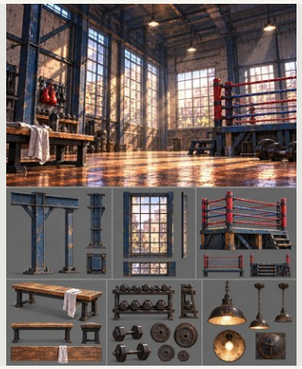
imagegen 모델링 시트와 표면 아틀라스 → UModeler X 편집 원본 → Blender 정리 → Unity FBX·재질·베이크 조명 순으로 구성했습니다. 원본과 런타임 출력을 나눠 보관했습니다.

체육관: 변형하지 않는 공간

공간·재질별 결합과 평면 분할 정리로 503개 메시/렌더러를 15개로, 20,400삼각형을 12,810개로 줄였습니다. 리그·애니메이션·Collider를 넣지 않고 정적 플래그와 UV2를 사용했습니다.

경기장: 구도와 레이어

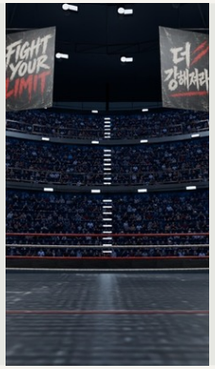
경기장 모델링 레퍼런스, 생성 아틀라스·관중·배너를 사용했습니다. 3D 배경을 직접 출력하고 선수·HUD와의 레이어·카메라 구성을 확인했습니다.



AI
생성한 체육관 모델링 레퍼런스



AI
생성한 체육관 표면 아틀라스



DCC
경기장 Blender 작업 미리보기



EDITOR
체육관 3D 환경 · 카메라 렌더

503 → 15
체육관 렌더러

20,400 → 12,810
Unity 삼각형

샌드백: 생성 시안에서 최적화된 물리 리그까지.

만드는 속도뿐 아니라 최종 사용 비용과 반응 방식을 검토했습니다.

제작과 최적화

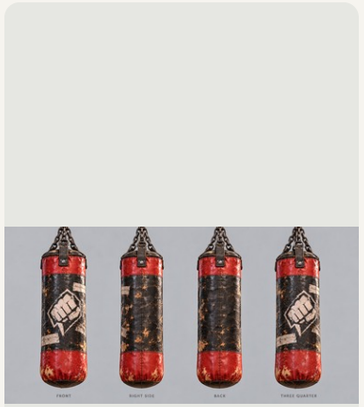
AI로 샌드백 레퍼런스와 가죽 텍스처를 만들고 UModeler X로 형태를 구성했습니다. Blender에서 리토폴로지·UV·베이크·5본 리그를 구성해 렌더러·메시·재질을 각각 1개로 정리했습니다.

물리 반응으로 전환

최종 버전은 애니메이션 클립과 Animator Controller가 없습니다. 고정 1개·동적 Rigidbody 3개와 ConfigurableJoint 3개가 충격에 반응하고, 본은 단위 스케일 프록시 회전을 따라갑니다.

실제 타격 이벤트와 연결

화면 타격 위치를 월드 접촉점으로 바꿔 힘을 전달하고 FixedUpdate에서 누적 임펄스를 적용합니다. 최종 샌드백은 별도 RenderTexture 없이 메인 카메라에서 직접 표시합니다.



AI
초기 샌드백 모델링용 AI 레퍼런스 · 외부 원본 백업



EDITOR
최초 UModeler X 샌드백 모델 · 제작 이력



EDITOR
최종 물리 리그의 치명타 자세 · 격리 PhysX 검증

62,800 → 3,208
삼각형 · 94.9% 감소

151 → 1
메시

0개
최종 애니메이션 클립

좋아 보이는 시안과
일관된 검증 조건 사이.

피드백을 반영해 표현 방식을 바꾸고 같은 조건으로 결과를 비교했습니다.

초기 결과의 판단

1차 만화풍 결과는 광택·색보정 변화가 크고 만화 표현은 부족했습니다. 노출·대비·채도·Bloom을 이전 기준으로 돌리고, 샌드백 표면의 3단 명암과 테두리 표현으로 방향을 조정했습니다.

플랫폼별 비용 분리

PC는 깊이+노멀로 외곽선을 찾고, Mobile은 깊이만 사용해 노멀 프리패스와 샘플을 생략합니다. 배경 흐림은 별도 배경 카메라에 적용하고 Overlay HUD의 선명도를 유지합니다.

통제한 비교

아래 PBR/셀 비교는 같은 조명·카메라·후처리에서 재질만 바꾼 결과입니다. PC Deferred와 Mobile Forward를 Editor에서 확인했으며 실제 모바일 GPU 성능은 미측정입니다.



EDITOR
BEFORE · 동일 조명·구도 · PBR 재질 비교 기준



EDITOR
AFTER · 동일 조명·구도 · 셀 재질 적용 후

손맛은 타이머와 데이터를 흔들지 않게.

게임 사건을 구독해 실행하고, 종료·중단까지 정리하는 연출을 만들었습니다.

연출의 독립성

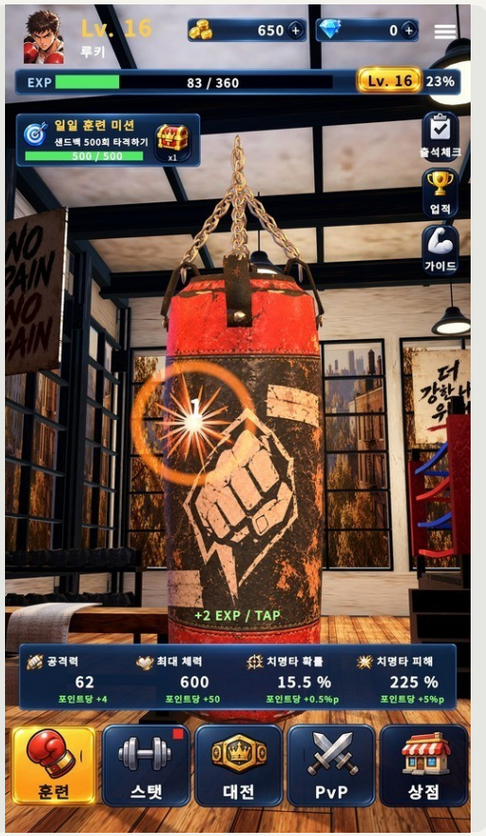
공격·체력·라운드 이벤트로 Feel 프리셋, DOTween 펀치·반동, 타격광·충격파·스파크, 피해 숫자와 지연 체력바를 연결했습니다. 전역 timeScale을 바꾸지 않고 unscaled time을 사용합니다.

재사용과 복원

VFX는 사전 배치한 8개 슬롯을 재사용합니다. 비활성화 때 오디오·잔상·이펙트를 정리하고, 종료·중단 때 카메라 화각을 원래 값으로 복원합니다.

검증으로 닫은 작업

기록상 30회 연타에도 슬롯 수가 유지되고 성장 데이터가 변하지 않았습니다. 샌드백은 격리 PhysX에서 일반·치명타·30연타·과부하·재활성화를 검사했습니다. 전체 Play 통합과 실기 성능 검증은 구분합니다.



PLAY
샌드백 직접 렌더 · 실제 Play 모드



EDITOR
Unity Game View · 승리 표시 이벤트 주입 검증

제작 탐색도
현재 결과와 구분해 남기다.

AI 레퍼런스 제작 과정과 최종 채택 범위를 설명합니다.

방향을 고정하는 이미지

적 복서의 정면·측면·후면과 가드 자세, 얼굴·글러브·쇼츠·부츠 상세 시트를 생성했습니다. 같은 캐릭터의 실루엣과 장비를 여러 시점에서 맞추기 위한 모델링 참고 자료입니다.

실험 결과의 선택

해당 시트 기반 모델링 탐색이 있었지만, 이후 모델·재질·프리팹·프리뷰 씬을 삭제하고 레퍼런스 2장만 남겼습니다. 따라서 이 페이지는 최종 전투 캐릭터 모델 구현 성과로 제시하지 않습니다.

실무에 가져갈 방식

요구사항과 입력 이미지를 기록하고, 산출물·실험·폐기 결과를 구분합니다. AI 작업에서도 무엇을 채택했으며 어떤 결과가 현재 빌드에 남는지 설명할 수 있도록 관리합니다.

만든 결과의 수보다, 채택한 이유와 현재 상태를 설명합니다.



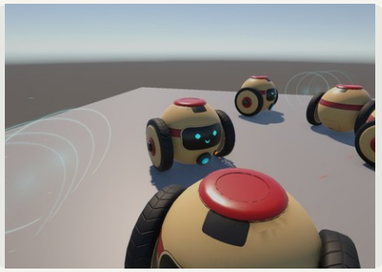
AI
적 복서 정면·측면·후면 레퍼런스 · 제작 탐색 기록



AI
적 복서 상세 레퍼런스 · 모델은 이후 삭제, 참고 이미지만 보존

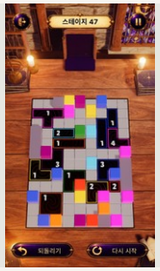
함께 일할 때
기여할 수 있는 일.

게임 클라이언트의 구현·도구·표현을 연결하는 역할을 맡고 싶습니다.



규칙을 구조로 정리합니다

Ability, 조건 평가, Domain/Repository, 서버 권한 경계를 사용해 변경 범위를 나눴습니다. 새로운 기능이 어느 계층에 들어가야 하는지 판단하고 설명합니다.



제작자의 반복 작업을 줄입니다

스토리 그래프, 에셋 저장 경로, 사운드 ID 생성·검증처럼 콘텐츠를 만드는 흐름도 함께 다룹니다. 공용 시스템과 개별 게임 기능을 연결합니다.



AI 결과를 실행 가능한 형태로 만듭니다

시안 생성 → 자산 분리 → 메시·재질·UI 연결 → 플랫폼별 비용 판단 → 검증으로 이어갑니다. 직접 맡은 부분과 협업한 부분, 측정한 값과 남은 검증을 구분합니다.

근거와 범위.

면접에서 설명할 수 있도록 코드·커밋·작업 기록을 함께 정리했습니다.

개발 시작일과 진행 상태

시작일은 Git의 도달 가능한 전체 로컬 ref에서 확인한 최초 author date를 기준으로 표기했습니다. 세 프로젝트 모두 현재 개발 중이며 종료일은 표기하지 않았습니다. 최초 기획일이나 연속 전일 투입 기간을 의미하지 않습니다.

개인 기여

사용자 확인: Lunch Royale 클라이언트 전체, 동료 AI 개발자 모델 학습 / Fighter Tap 전체. Puzzle Zip은 wooing26·LikeDev_PWY 커밋의 실제 변경 파일로 개인 기여를 확인했습니다. 커밋 개수를 기여율로 환산하지 않았습니다.

화면과 검증의 종류

PLAY: Editor Play 캡처 / EDITOR: 카메라 렌더·Canvas 합성·표시 이벤트 검사 / AI: 생성 이미지 / REF: 제공 레퍼런스 / ASSET: 프로젝트 자산. Editor Mobile 품질은 실제 기기 캡처가 아닙니다.

함께 제공하는 자료

HTML 열람본은 이미지 확대와 출처를 지원합니다. evidence-index.md에는 근거 파일·커밋과 확인 범위, assets-manifest.json에는 원본 경로·해시·이미지 분류를 기록했습니다.

박우영 · wyp1226@naver.com

근거 01 / Lunch Royale

앞선 페이지의 근거 ID에 대응하는 코드·기록과 확인 범위입니다.

GIT · 개발 시작일과 작성자 산정

research/*-git-history.tsv; research/minigame-contributions.txt
시작일은 도달 가능한 전체 로컬 ref의 최초 author date 기준. 세 프로젝트 모두 사용자 확인상 현재 개발 중이며 종료일은 표기하지 않음. 최초 기획일/연속 투입 기간과 다름.

LR1 · Lunch Royale · 현재 행동 AI 경계

Assets/02. Scripts/02-03. Agent/BattleAgentHub.cs; Ability/ObservationAbility.cs; Ability/MoveAbility.cs; Ability/PushAbility.cs
현재 코드와 lunch_agent.onnx. 모델 학습은 동료 AI 개발자 담당.

LR2 · Lunch Royale · 판정과 재시도

Assets/02. Scripts/02-05. GameMode/GameResultEvaluator.cs; ConditionGroup.cs; Condition/TimeLimitCondition.cs; Assets/02. Scripts/02-04. Ingame/AgentSpawner.cs
동시 탈락 수집, 다음 Update 평가, Training/Gameplay 경계.

LR3 · Lunch Royale · 시각 피드백

Assets/02. Scripts/02-03. Agent/Ability/FaceDisplayAbility.cs; Assets/02. Scripts/02-03. Agent/VfxController.cs
MaterialPropertyBlock, 이벤트 기반 표정, 취소 토큰.

LR4 · Lunch Royale · 생성형 AI 협업 대화

01a031ff-07f7-76e1-8d82-77deed34d206; 019fff06-af5d-73d3-a15f-34b2a9767e00; 019f20ab-5992-7661-b365-bf4adc4cf22a
콘셉트 수정·분리 FBX·관찰값 및 할당 개선 논의. 현재 코드와 재대조. 오래된 생성 콘셉트 PNG는 현 위치에서 찾지 못해 수록하지 않음.

LR5 · Lunch Royale · Google Sheets 준비 계층

Assets/02. Scripts/02-02. GoogleSheets/GoogleSheetService.cs; GoogleSheetSelectionManager.cs; GoogleWorkbook.cs
XLSX 다운로드와 선택 계층은 존재. 현재 프로젝트에 GoogleScene은 없으므로 경기 참가자 연결 완료로 주장하지 않음.

LR6 · Lunch Royale · 이번 화면 확보

research/lunch-play-1.png; lunch-play-2.png; lunch-closeup.png
2026.10.22 MainScene Play. frameCount 236→246, 8개 Agent, BattleBehavior/lunch_agent 연결 확인. 근접 캡처만 카메라 변경. 종료 후 원래 빈 씬·배경 실행 false 복원.

근거 02 / Puzzle Zip

앞선 페이지의 근거 ID에 대응하는 코드·기록과 확인 범위입니다.

PZ1 · Puzzle Zip · 개인 기여 Git 근거

f6f3693d; 868be0af; e78f8d4c; 3f7e6f67; f4786e35; 164f2ebf; 872fc244
wooing26 / LikeDev_PWY 작성자. JellyMelt, 저장, 스토리 편집기, 오디오, 광고. PotionSort 주 구현은 다른 작성자.

PZ2 · Puzzle Zip · 퍼즐 생성과 도구

Assets/02. Scripts/02-02. Games/JellyMelt/01. Map/MapBuilder.cs; 03. Entity/Tile/PlayableTile.cs; Assets/02. Scripts/02-08. Story/Editor/StoryGraphWindow.cs; StoryGraphDataEditor.cs
풀링·동적 맵·스토리 노드 편집·Undo·저장 구조.

PZ3 · Puzzle Zip · 저장 계층

Assets/02. Scripts/02-05. PlayerDataStore/00. Core/03. Runtime/PlayerDataStore.cs; 02. Repository/LocalRepository.cs; ServerRepository.cs
Domain/DTO/Repository 분리. 현재 ServerRepository도 임시 로컬 JSON 구현이며 클라우드 저장 완료로 표현하지 않음.

PZ4 · Puzzle Zip · 환경 제작과 검증

output/jellymelt-environment-v2/README.md; imagegen-prompts.json; validation-summary.json
UModeler X → Blender → 개별 소품/부품 구조. 32개 카메라 검사와 모바일 텍스처 설정.

PZ5 · Puzzle Zip · AI UI와 최종 스타일

output/imagegen/jellymelt-ui-v1/design-notes.md; output/imagegen/jellymelt-ui-parts-v1/prompts.md; output/jellymelt-magic-style/README.md; ui-framing-matrix.json
AI 목업과 실제 Editor 합성 렌더 구분. 투명 PNG 10개. 24개 프레이밍 검사. 되돌리기/일시정지 기능 미연결.

PZ6 · Puzzle Zip · 사용자 제공 레퍼런스

C:/Users/Wooeng/Downloads/꼬마 마법사의 블록 퍼즐-1.png; 꼬마 마법사의 물약 정렬 퍼즐-2.png
비주얼 방향 참고. 캐릭터·수치는 실제 구현 범위와 다를 수 있음.

근거 03 / Fighter Tap

앞선 페이지의 근거 ID에 대응하는 코드·기록과 확인 범위입니다.

FT1 · Fighter Tap · 게임·네트워크·저장

Docs/GameDesign.md; Assets/02. Scripts/02-04. Fighter/Network/FighterNetworkAbility.cs; Assets/02. Scripts/02-05. Battle/Network/PvpMatchNetwork.cs; Assets/02. Scripts/02-03. Data/Logic/ProfileStore.cs
서버 권한 경로와 프로필 복구. 2기기 PvP는 미검증.

FT2 · Fighter Tap · 규칙 검증

Docs/Validation.md; Assets/02. Scripts/02-10. Tools/Editor/FighterChecks.cs
2026.10.16 당시 28개 규칙·5개 네트워크 단위 검사 통과 기록. 이후 아트/물리 최종본의 전체 재검증과 구분.

FT3 · Fighter Tap · AI 레이어 제작

Docs/SceneArtwork/README.md; Assets/04. Images/04-03. Title/README.md; Assets/04. Images/04-04. SceneArtwork/README.md
배경/캐릭터/UI 분리. Game View 이벤트 주입 화면은 실제 경기 결과 캡처와 구분.

FT4 · Fighter Tap · 체육관·경기장 제작

Docs/LobbyGym/README.md; Docs/Arena/README.md; Docs/LobbyGym/reference-prompt.txt; atlas-prompt.txt
체육관 503→15 렌더러, 20,400→12,810 삼각형. 실기 성능 수치 아님.

FT5 · Fighter Tap · 샌드백 최종 버전

Docs/TrainingBagMobile/README.md; optimization-report.json; physics-validation.json; Docs/TrainingBag3D/README.md
62,800→3,208 삼각형, 물리 리그, 애니메이션 없음. 격리 Editor PhysX 검증.

FT6 · Fighter Tap · 만화풍 렌더링

Docs/LobbyManga/README.md; Docs/LobbyBloom/README.md
최종 셀 표면, PC 깊이+노멀/모바일 깊이 외곽선. 같은 조명·카메라 비교.

FT7 · Fighter Tap · 타격 연출

Docs/CombatPresentation/README.md; Assets/02. Scripts/02-07. Stage/CombatFeedbackController.cs; CombatCameraZoom.cs
8개 VFX 슬롯, unscaled time, 화각 복원·30연타 검증.

FT8 · Fighter Tap · 보존된 적 레퍼런스

Assets/05. Models/EnemyBoxer/References/EnemyBoxer_Turnaround.png; EnemyBoxer_Details.png; 대화 01a15703-db64-7422-8df1-4894f2d63c21
적 모델 삭제 요청 반영. 참고 이미지 2장만 현재 보존. 완성 모델/전투 연결 성과로 세지 않음.